BUILD GUIDE

Conclave, built module by module

The full sequence for building the control plane where models, agents, workflow engines and tools work as one governed team: every package, connection, library, design technique and screen, in the order to build them, followed by what to build next.

30	5	13	3
build stages, in order	core packages	modules and docks	apps: server, web, bridge

Written against the repository as it stands on 10 November 2026. Stages 1 to 29 describe what exists and how to rebuild it. The rest of Part D is what to build next; rows already built are marked IN REPO.

Contents

Part A · Before you start

A1 What Conclave is, and what makes it different

A2 The vocabulary you must keep straight

A3 Architecture in one page

A4 Design techniques used everywhere

A5 Rules for every stage

Part B · Foundation

Stage 1 Monorepo and toolchain

Stage 2 Contract package

Stage 3 Database layer

Stage 4 Durable execution engine

Stage 5 Module SDK and conformance kit

Stage 6 Secret vault

Stage 7 Accounts, roles and access

Stage 8 Module registry and connections

Stage 9 First modules: mock, OpenAI-compatible, Anthropic

Stage 10 Workspaces, templates and versions

Stage 11 Budgets and the usage ledger

Stage 12 Orchestrator: runs, steps, approvals

Stage 13 Events, outbox and realtime

Stage 14 Inbox and notifications

Stage 15 API server

Stage 16 Web app: shell and core screens

Part C · Composition and interoperability

Stage 17 Node editor

Stage 18 Tool port and the MCP dock

Stage 19 Branching, connectors and loops

Stage 20 Workspace steps (sub-tasks)

Stage 21 Agent and Workflow ports

Stage 22 Module protocol and the docks

Stage 23 Local Bridge

Stage 24 A2A dock

Stage 25 Installed modules

Stage 26 Trace

Stage 27 Insights and the audit trail

Stage 28 Tests, CI, accessibility, packaging

Part D · What to build next

Stage 29 Governance across nested calls

Stage 30 The remaining roadmap, in order

D3 UI improvements still needed

D4 Gates: how to know it works

Appendices

X1 Every library and why

X2 Database tables

X3 API routes

X4 Settings

X5 Build checklist

PART A

Before you start

A1 · What Conclave is, and what makes it different

Conclave is a control plane. It does not try to be the model, the agent or the workflow engine. It connects systems that already exist, lets a person compose them into larger tasks, and governs and records the whole run from one place.

The promise to build towards: **connect anything that can do work, expose data or raise an event; compose those capabilities into larger systems; control, observe and govern the whole execution from one place.**

The six kinds of user in the specification (creator, researcher, developer, operations team and so on) are examples and templates. They are not architecture. One set of primitives serves all of them.

Existing product type	What it already does well	What Conclave does instead
Agent organisers (Paperclip and similar)	Agents as employees: roles, delegation, budgets, approvals	Agents are one kind of node among models, tools, workflow engines and, later, knowledge and devices, all behind one call contract
AI app builders (Dify and similar)	Visual workflows, knowledge pipelines, plugins, publishing, inside their own runtime	Orchestrates outside systems where they already run
Durable execution (Temporal)	Retries, timers, child workflows, signals, recovery	Sits above it. The engine is behind an interface; do not rebuild Temporal
Automation tools (n8n, Zapier, Make)	Integrations, triggers, branching	Each is a Workflow-port node, called through a webhook dock
MCP and A2A	Standard access to tools; standard talk between agents	Adapters into ports. Never compete with them

"We connect anything" is therefore not enough. The difference is five things together:

1. **Capability-level interoperability**: unlike systems fit one typed port each.
2. **One invocation and execution model**: the same envelope, errors, retries and events for every system.
3. **Limits that hold across nested calls**: permissions and budgets only ever narrow as work is delegated.
4. **One history**: a single trace and a single ledger for a run that crosses systems and workspaces.
5. **Portable workspaces**: a composition can be exported, imported and reused.

A2 · The vocabulary you must keep straight

Most architectural trouble in a platform like this comes from blurring these words. Keep each one a separate thing in code.

Word	Meaning	Example
Port	What capability is needed. A typed, versioned interface	`agent@1`
Protocol	How the talking happens on the wire	A2A JSON-RPC, MCP, CMP
Module	The adapter that implements a port for some system	`conclave/a2a`

Dock	Where the module's code runs	built-in, protocol, remote, sidecar, bridge
Connection	One configured instance of a module, with its own secrets and limits	"Research agent (staging)"
Node	One usable capability a connection exposes	a model, a tool, a skill, a workflow
Role	A named seat in a workspace, filled by a node	"Researcher"
Workflow / task type	A composition of steps over roles	"Research brief"
Run / task	One execution of a task type	"Tidal energy in the North Sea"
Step	One attempt at one piece of a run	"Draft, attempt 2"

Installing a module does not create one node. Three connections to three gateways of the same module give three sets of nodes, each with its own permissions and spend limit.

A3 · Architecture in one page

Experience
Web app (React): Home, Inbox, Workspaces, node editor, Trace, Insights, Connections, Settings · REST API · access tokens · Local Bridge CLI

Control plane · `packages/core`
Accounts and access · module registry · connections and secrets · workspaces · budgets and ledger · inbox and notifications · insights (trace, usage, audit)

Orchestrator
Plans the DAG, runs steps, branches, loops, approvals, sub-tasks, outside runs

Engine · `packages/engine`
One leased task per run, signals, timers, crash recovery, in Postgres

Module layer · `packages/module-sdk`
Manifest · `defineModule` · conformance kit · Conclave Module Protocol (JSON-RPC over HTTPS or stdio) · bridge protocol

Protocol docks
OpenAI-compatible, MCP, A2A, webhook

Built-in
Anthropic, mock, installed modules

Remote
CMP over signed HTTPS

Sidecar
CMP over stdio

Bridge
A person's computer

Outside systems
Model providers · agents · workflow engines · tool servers · later: knowledge, channels, devices, storage

The rule that holds it together: **core knows ports and never names a product.** A script (`scripts/lint-core-names.mjs`) fails the build if a product name appears in core.

DEPENDENCY ORDER OF THE PACKAGES

```
contract  →  db  →  engine  →  module-sdk  →  core  →  server  →  web
                                    |                      ^
                                    +→  modules/*  ————————+      bridge (uses module-sdk only)
```

Nothing lower may import anything higher. `contract` has no knowledge of the database; `module-sdk` has no knowledge of core. That is what lets a module author depend on two small packages only.

A4 · Design techniques used everywhere

Technique	Where, and why
Ports and adapters (hexagonal)	Ports in `contract`, adapters in `modules/*`. Core is testable with a mock module and never changes when a system is added
Contract-first, shared types	One package of types and zod schemas used by server, web and modules, so the API cannot drift from the UI
Durable state machine	A run is rows in Postgres. A worker "advances" it under a lease; anything in flight can be redone after a crash
Lease with `SKIP LOCKED`	Exactly one worker advances a run at a time, without a queue service
Signals	Approvals, agent events and callbacks are written as rows and applied by the run's own worker, so there are no races
Transactional outbox	Every state change writes its event in the same transaction; a relay publishes it. No event without the change, no change without the event
Reserve, then settle	Money is held before a call and settled to the real cost after. A budget is never found out to be blown afterwards
Idempotency keys	`run:step:attempt` to providers; an `Idempotency-Key` header on POSTs that start work
Envelope encryption	Master key wraps per-organisation data keys; data keys encrypt secrets with AES-256-GCM. Destroying a data key shreds its data, backups included
Default deny	Module egress limited to manifest hosts; sidecars off unless listed; bridge targets added only on the device; every API route must declare a guard
Capability narrowing	A child call gets a subset of its parent's permissions and a share of what is left of its budget
Normalised errors	Eleven error codes with a `retryable` flag, so retry behaviour is the same for every system
Expand-then-contract migrations	Forward-only SQL; add first, remove later, so a rolling deploy never breaks
Tenant column everywhere	Every row has `org_id` and every query filters on it
Conformance over trust	A module is accepted because it passes the kit, not because of who wrote it
One database for two profiles	A small query interface over PGlite (no server needed) and node-postgres. The same SQL runs on both, and tests run on both
W3C trace context	Every outside call carries a `traceparent` whose trace id is the top task

A5 · Rules for every stage

- **Build in the order given.** Each stage only uses what earlier stages made.
- **Finish means tested.** Each stage ends with a "Done when" box. Write those tests before moving on.
- **Keep the three checks green:** `npm run typecheck`, `npm test`, `npm run lint:core-names`.

- **Do not add a port, a dock or an engine until a real integration needs it.** Each one is a contract, a test suite and a security surface.
- **Audit before rebuilding.** If the repository already has a stage, check it against this guide and fill gaps; do not rewrite it.

PART B

Foundation

Stages 1 to 16 give a working product: sign in, connect a model, make a workspace, run a task with a budget, approve it, and watch it live.

STAGE 1 Monorepo and toolchain IN REPO

> **Goal.** One repository where every package is TypeScript source, with no build step between packages, and one command each for tests and type checks.

Depends on	Nothing
Libraries	Node 22 or newer · npm workspaces · TypeScript 6 · tsx 4 (runs TypeScript directly) · Vitest 5
Files	`package.json`, `tsconfig.base.json`, `tsconfig.json`, `vitest.config.ts`, `.gitignore`, `.dockerignore`

BUILD STEPS

1. Root `package.json`: `"type": "module"`, `"private": true`, `engines.node ≥ 22`, workspaces `packages/*`, `modules/*`, `apps/*`.
2. Every workspace package exports its source: `"exports": { ".": "./src/index.ts" }`. The server runs with `node --import tsx`, so nothing is compiled between packages. This keeps the edit-run loop short and removes stale-build bugs.
3. `tsconfig.base.json`: strict mode, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, ES2023 target, bundler module resolution. Strictness is cheap now and expensive to add later.
4. Root `tsconfig.json` includes `packages/*/src`, `modules/*/src`, `apps/server/src`, `apps/bridge/src`. The web app has its own tsconfig because it needs DOM types and JSX.
5. `vitest.config.ts`: include `**/*.test.ts` under packages, modules, server and bridge; 30 s timeouts; `pool: "forks"` because each test file starts its own in-memory database.
6. Scripts: `dev:server`, `dev:web`, `start`, `build:web`, `test`, `typecheck` (two `tsc` runs), `lint:core-names`, `test:pg`, `test:a11y`.
7. Write `scripts/lint-core-names.mjs` now: it scans `packages/core/src` and fails if a product name (a provider, an agent product, a workflow tool) appears. Running it from day one is what keeps core generic.

> **Done when.** `npm install`, `npm test` and `npm run typecheck` pass on an empty package.

STAGE 2 Contract package IN REPO

> **Goal.** The shapes everything else agrees on: the call envelope, the ports, the workflow schema, events and API types. This is the most important package to get right, because every other one depends on it.

Depends on	Stage 1
Libraries	zod 4 (schemas and validation) · uuidv7 (ids that sort by time)
Files	`packages/contract/src/`: `call.ts`, `ports.ts`, `workflow.ts`, `conditions.ts`, `events.ts`, `api.ts`, `index.ts`

BUILD STEPS

1. **Ids.** Export `newId()` from uuidv7. Time-ordered ids give free ordering, good index locality, and paging by id.
2. **The call envelope** (`call.ts`). Every node, whatever serves it, receives a `NodeCall` and answers with a `NodeResult` .

```
interface NodeCall {
  callId; runId; parentCallId; depth;        // where in the nest
  nodeId; input: { text?, data?, files? };   // what to do
  outputSchema?;
  budget: { usd, tokens?, deadline? };       // most it may spend
  permissions: string[];                     // "call:node:<id>"; children get a subset
  trace: { traceparent };                    // W3C, trace id = top task
  idempotencyKey;                            // run:step:attempt
}
```

3. **Errors.** Eleven codes: `auth, rate_limited, invalid_request, not_found, timeout, unavailable, refused, cancelled, budget_exceeded, loop_detected, internal` . A `NodeCallError` class carries the code and `retryable` (true by default for rate_limited, timeout, unavailable). `toNodeError(err)` maps anything thrown, including `AbortError` to `cancelled` .
4. **Usage and cost source.** `Usage { inputTokens, outputTokens, costUsd, costSource }` where the source is `metered | reported | reconciled | estimated` . Recording where a number came from is what later lets Insights say how exact the spend is.
5. **Trace helpers.** `traceIdOf(rootRunId)` and `traceparent(rootRunId, callId)` . The span id uses the last 16 hex characters of the call id, because the start of a UUIDv7 is a timestamp and could repeat.
6. **Ports** (`ports.ts`). List all twelve names so manifests can declare them (`model, agent, workflow, tool, knowledge, trigger, channel, device, storage, media, identity, secrets`), but give interfaces only to the ones you implement. A `PortRef` is `name@major` . `CallContext` is `{ call, signal, emit }` .
   - `ModelPortV1` : `listModels()` , `generate(req, ctx)` .
   - `ToolPortV1` : `listTools()` , `call(name, args, ctx)` . Tools carry `sensitive` .
   - `AgentPortV1` : `listSkills` , `startRun` , `streamEvents` , `provideInput` , `cancel` . Add in Stage 21.
   - `WorkflowPortV1` : `listWorkflows` , `trigger` , optional `poll` , `callback` , `cancel` . Add in Stage 21.
7. **Workflow schema** (`workflow.ts`). A zod discriminated union of steps by `kind` . Start with `role` and `approval` ; later stages add `tool, branch, classify, merge, text, stop, workspace, agent, automation` . Shared step fields: `key` , `needs` (the DAG), `optional` , `disabled` , `timeoutMs` , `retry` . Also export `validateBrief` , `edgesOf` , `downstreamOf` , `portForStep` .
8. **Events** (`events.ts`): `run.updated` , `step.updated` , `step.delta` , `inbox.updated` , `notification` , each in an envelope with a per-instance `seq` and topics (`user:<id>:inbox` , `workspace:<id>:runs` , `run:<id>`).
9. **API types** (`api.ts`): every DTO the REST API returns. The web app imports these, so a changed field is a compile error in the UI.

DESIGN NOTES

- Standardise the envelope, not each system's behaviour. A model, an agent and a workflow engine keep their own port; they share the call, the error and the usage.
- A port must not force meaningless methods on a module. That is why agents get `startRun` and events while a tool only gets `call` .
- Handles returned by long-running ports must be plain JSON, because they are stored in a checkpoint and read back after a restart.

Done when. `workflow.test.ts` covers schema parsing, cycle detection and brief validation. No file in this package imports anything but zod and uuidv7.

STAGE 3 Database layer IN REPO

Goal. One small query interface that works the same on an embedded database and on a Postgres server, plus migrations.

Depends on	Stage 1
Libraries	`pg` 8 (node-postgres) · `@electric-sql/pglite` (Postgres compiled to WebAssembly, for the Lite profile and tests)
Files	`packages/db/src/index.ts`, `migrations.ts`

BUILD STEPS

1. Define `Queryable { query<T>(sql, params) }` and `Db extends Queryable { kind, tx(fn), close() }`. Core writes plain parameterised SQL against this. No ORM: the SQL is the same on both backends, and locks (`for update`, `skip locked`) stay visible.
2. `createDb(url)`: `pglite:memory`, `pglite:<dir>`, or `postgres://`. For Postgres use a pool of 10 and an optional schema set through `search_path` on every connection.
3. `createTestDb()`: in-memory PGlite by default; when `CONCLAVE_TEST_DATABASE_URL` is set, a fresh schema per test on real Postgres, dropped on close. This is how the same suite runs on both.
4. Helpers: `num()` (numerics arrive as strings), `iso()`, `json()` (send JSON as text and cast with `::jsonb`).
5. `migrate(db)`: a `migrations` array of `{ id, sql }`, applied in order inside a transaction, recorded in a table. Forward-only. Never edit a migration that has shipped; add a new one.
6. Money is `numeric(14,6)`, never a float. Times are `timestamptz`. Every tenant table has `org_id`.

The tables themselves are added by the stage that needs them; Appendix X2 lists them all with the migration they belong to.

Done when. A test migrates an in-memory database, runs a transaction that rolls back on a throw, and the same test passes with `npm run test:pg`.

STAGE 4 Durable execution engine IN REPO

Goal. Guarantee three things and nothing more: only one worker advances a run at a time; work survives a crash; timers and signals wake runs. Workflow meaning stays in core.

Depends on	Stages 2, 3
Libraries	None beyond the db package
Files	`packages/engine/src/index.ts`
Tables	`engine_tasks` (one "advance" row per run: `available_at`, `locked_until`, `locked_by`, `generation`, `attempts`), `engine_signals`

BUILD STEPS

1. The interface:

```
interface ExecutionEngine {
  wake(q, runId, at?)                 // call inside the caller's transaction
  signal(q, runId, name, payload)     // deliver and wake
  forget(q, runId)
  startWorker(handler, opts): EngineWorker
}
type AdvanceHandler = (runId, signals, heartbeat) ⇒ Promise<AdvanceOutcome>
type AdvanceOutcome = { kind: "idle" } | { kind: "reschedule", at: Date }
```

2. **Claim.** One statement selects a due task `for update skip locked`, sets `locked_until = now() + lease` and `locked_by`. Expired leases are due again, which is the whole crash-recovery story.
3. **Heartbeat.** The handler calls `heartbeat()` during long steps to extend the lease.
4. **Generation.** `wake` bumps `generation`. After the handler returns "idle", delete the task only if the generation is unchanged; otherwise something woke the run meanwhile and it must run again. This closes the lost-wake-up race.
5. **Signals.** Read unconsumed signals when claiming, pass them to the handler, mark them consumed in the same transaction as the handler's effects.
6. **Worker.** A polling loop with a concurrency limit; `tick()` and `drain()` for tests (`autoStart: false`), so tests are deterministic and need no sleeps.

DESIGN NOTES

- This is deliberately the smallest engine that is correct. It needs only Postgres, which is what the Lite profile promises.
- Keep it behind the interface. When scale needs Temporal, write a second implementation; core does not change. Do not make a queue such as BullMQ the only engine: it would tie Lite to Redis.
- Define execution meaning once, in core, independent of the engine: at-least-once step execution, made safe by idempotency keys and by recording each attempt.

> **Done when.** Tests show: two workers never run one run at once; a task whose lease expired is picked up; a wake during a handler causes one more advance; a signal reaches the handler exactly once.

STAGE 5 Module SDK and conformance kit IN REPO

> **Goal.** Everything a module author needs, in one small package: how to declare a module, what it is given, and a test kit that says whether it behaves.

Depends on	Stage 2
Libraries	zod 4
Files	`packages/module-sdk/src/`: `manifest.ts`, `index.ts`, `conformance.ts` (later: `cmp.ts`, `bridge-protocol.ts`)

BUILD STEPS

1. **Manifest schema** (zod). The install screen, the egress allow-list and the secret store are all driven by it.

```
id          "publisher/name"
version     semver
name, description
ports       ["model@1", ...]            at least one
docks       ["built-in"|"protocol"|"remote"|"sidecar"|"bridge"]
config      [{ key, label, type: string|url|secret|number, required, placeholder, help,
default }]
permissions { egress: [host | "{config.key}" | "{config.key[]}"], secrets: [key] }
sensitiveActions, outsideVersions, nodeUi { icon, colour }, reference
```

2. `defineModule({ manifest, create })` validates the manifest at load time, so a mistake stops the server at start, not at first use.
3. `ModuleContext`, given to `create` once per connection: non-secret `config`, decrypted `secrets` for this connection only, an egress-checked `fetch`, a `log`, and the `connection { id, orgId }`. A module must use `ctx.fetch`, never the global one.
4. `ModuleInstance`: `ports`, `health()`, optional `outsideVersion()`, `discover()`, `dockedModule()`, `close()`.
5. **Helpers**: `httpError(status, message)` maps HTTP statuses to contract errors; `statusForError` maps errors to a connection status; `requireString`; `secretKeys(manifest)`; `declaredPorts(manifest)`.
6. **Conformance kit** (`runConformance(module, options)`). For each declared port it checks: lists return the right shape; a call completes; errors use contract codes; cancellation through the signal works; for long-running ports, handles survive a JSON round trip and watching a finished run again yields its final event. It returns named checks with pass or fail, and takes a fake `fetch` so tests need no network.

Done when. A ten-line module passes the kit, and a module with a bad id fails at `defineModule` with a readable message.

STAGE 6 Secret vault IN REPO

Goal. Keys are encrypted at rest, never returned by the API, and can be destroyed for good.

Depends on	Stage 3
Libraries	`node:crypto` only
Files	`packages/core/src/secrets.ts`
Tables	`data_keys` (per organisation, wrapped), `secrets` (ciphertext)

BUILD STEPS

1. A 32-byte master key from `CONCLAVE_MASTER_KEY`, or generated into `<data dir>/master.key` with mode `0600`.
2. Each organisation gets a random data key, stored wrapped by the master key.
3. `store(q, orgId, values)` encrypts a JSON object of secret values with the data key using AES-256-GCM (random 12-byte nonce, auth tag kept) and returns the row id. `open(q, secretId)` decrypts.
4. `seal` and `unseal` for secrets that belong to a person, not an organisation (a TOTP seed).
5. `shred(orgId)` marks the data key destroyed. Everything it encrypted is unreadable, including copies in backups.
6. Rule for callers: decrypt for one call, keep the values out of logs, errors and responses.

Done when. Tests show a round trip, that tampered ciphertext fails, and that after `shred` nothing can be read.

STAGE 7 Accounts, roles and access IN REPO

Goal. Know who is calling, in which organisation, and what they may do, in a workspace or outside one.

Depends on	Stages 3, 6
Libraries	`node:crypto` (scrypt for passwords, SHA-256 for token hashes) · `@simplewebauthn/server` (passkeys) · `qrcode` (TOTP setup)
Files	`packages/core/src/` : `errors.ts` , `access.ts` , `accounts.ts` , `mfa.ts`
Tables	`orgs, users, memberships, identities, sessions, api_tokens, invites, workspace_members, user_totp, recovery_codes, passkeys, auth_challenges, audit_log`

BUILD STEPS

1. **Errors.** `ConclaveError(status, title, detail, errors)` with helpers `notFound` , `invalid` , `conflict` , `budgetBlocked` (402). The server turns these into RFC 9457 problem details.
2. **Actor and Principal.** `Actor { orgId, userId }` for services; `Principal` adds `role` , `scopes` , `via: session | token` .
3. **Roles and permissions.** Four roles, each a superset of the one before:

Role	Adds
Viewer	`workspace:read` , `connection:read`
Runner	`task:run` , `approval:decide`
Editor	`workspace:edit` , `connection:manage`
Owner	`member:manage` , `org:manage`

4. `Access.workspaceRole(q, principal, workspaceId)` returns the per-workspace override if there is one, the organisation role otherwise, "owner" for a person's own personal workspace, and null for someone else's. `ensure()` throws 404 when a workspace is invisible and 403 when the permission is missing. A token's scopes narrow further; a token can never do more than its maker's role.
5. **Accounts.** Owner setup; email and password; sessions (a random cookie value, only its hash stored, 30-day expiry); invites (hashed token, optional fixed email); personal access tokens with a `cnc_` prefix and scopes; switching organisation; allowed sign-up domains.
6. **Outside sign-in.** `signInWithIdentity(provider, subject, email)` links by provider and subject, and by verified email if allowed. The provider code lives in the server (Stage 15).
7. **Second factors.** TOTP with single-use recovery codes (hashed), replay protection by remembering the last accepted step; passkeys through WebAuthn for sign-in and as a second step. A correct password with a second factor returns a short-lived ticket, not a session.
8. **Audit.** `audit(q, actor, action, target, detail)` writes one row. Call it in the same transaction as the change, for every change. The reading side comes in Stage 27.

Done when. `accounts.test.ts` and `mfa.test.ts` cover setup, sign-in, invites, token scopes, role overrides, private workspaces, TOTP replay and recovery codes.

STAGE 8 Module registry and connections IN REPO

> **Goal.** Turn "a module is installed" into "a configured, tested, limited connection whose nodes can be listed and called".

Depends on	Stages 5, 6, 7
Files	`packages/core/src/modules.ts`, `connections.ts`
Tables	`connections` (name, module id and version, dock, config, secret id, status, spend limit, disabled, later: ports and remote manifest)

BUILD STEPS

1. `ModuleRegistry.install(module, source)`, `get(id)`, `has(id)`, `list()` returning `ModuleDto` with config fields, permissions and source ("built-in" or "installed").
2. **Egress.** `allowedHosts(manifest, config)` resolves `{config.key}` to the host of that URL setting and `{config.key[]}` to a comma-separated list. `egressFetch` wraps `fetch` and refuses any other host with a message that names the module and the host.
3. `ModuleRuntime`. One live instance per connection, keyed by a stamp of its config; rebuilt when the connection changes and closed on eviction. Caches model, tool, skill and workflow lists for ten minutes.
4. `Connections` **service.**
   - `create`: validate values against the manifest's config fields (required, URL shape, number), split secret fields into the vault, store the rest as config, write an audit row.
   - `update`, `remove` (refused while a role uses it), `setDisabled` (the kill switch).
   - `test`: build the instance, call `health()` and `discover()`, save status, detail and outside version.
   - `models`, `tools`, `skills`, `workflows`: the nodes a connection exposes.
   - `instance(actor, id)`: what the orchestrator calls to get a live module.
   - `checkAll`: periodic health checks.
5. **DTO.** Never return secrets. Return `secretKeys` (which fields are set), month spend, and which workspaces use the connection.

CONNECTIONS TO OTHER PARTS

Workspaces point roles at connections. The orchestrator asks for instances. Budgets read a connection's spend limit. The trace and Insights join on connection id.

> **Done when.** `connections.test.ts`: secrets never appear in a DTO; a module that fetches an undeclared host is refused; a disabled connection cannot be called; audit rows exist.

STAGE 9 First modules: mock, OpenAI-compatible, Anthropic IN REPO

> **Goal.** Prove the Model port with three very different adapters, and get a free model for tests and first-run.

Depends on	Stage 5
Libraries	`@anthropic-ai/sdk` for the Anthropic module. The other two use only `ctx.fetch`
Files	`modules/mock-model`, `modules/openai-compatible`, `modules/anthropic`, each with `src/index.ts` and `src/index.test.ts`

BUILD STEPS

1. **Mock model** (built-in dock). Deterministic output from the prompt, a few named models with fixed prices, text deltas through `ctx.emit`, and prompt keywords that force a failure or a refusal, so retry and error paths can be tested without a network. No key, no cost.
2. **OpenAI-compatible** (protocol dock). Config: base URL, optional key, prices per million tokens. `listModels` from `/models`; `generate` streams `/chat/completions` over server-sent events, emits deltas, reads reported usage, and estimates usage by characters when the server reports none (marking `usageSource: "estimated"`). Egress is `{config.baseUrl}`. One module covers OpenAI, Ollama, vLLM, LM Studio, OpenRouter and Groq.
3. **Anthropic** (built-in dock, official SDK). Streaming, effort levels, prompt-cache token counts, mapping of refusals to the `refused` code, and passing the idempotency key. Before writing it, read the current SDK documentation for model ids and parameters; do not rely on memory.
4. Each module's test runs `runConformance` against a recorded or fake server, then a few specific checks (headers sent, usage read, errors mapped).

> **Done when.** All three pass the conformance kit. This is the first proof that the port works across unlike systems.

STAGE 10 Workspaces, templates and versions IN REPO

> **Goal.** A workspace holds roles (seats filled by nodes), task types (workflows), a budget and a version history, and can be exported and imported.

Depends on	Stages 2, 7, 8
Files	`packages/core/src/`: `workspaces.ts`, `templates.ts`, `builder.ts`, `render.ts`
Tables	`workspaces, workspace_versions, roles, task_types`

BUILD STEPS

1. **Roles.** `key, name, description, instructions, port, connection_id, model, effort`. A role is on one port; `model` holds the node name (a model, a tool, a skill or a workflow id).
2. **Task types.** `key, name, input_fields, workflow`. Input fields define the brief form.
3. **Three ways to create.** From a built-in template (start with Research Brief: search, research, draft, approval); from the guided builder (`composeWorkspace(answers)`: the same answers always give the same workspace, and `compose` previews without creating); blank; or by importing a package.
4. **Saving a task type** validates everything: the schema, unique keys, that `needs` point at real steps, no cycles, roles exist and are on the right port, templates reference real fields and earlier steps. Error messages name the step. Each save bumps the workspace version and stores the manifest in `workspace_versions`.
5. **Runs keep their workflow.** A run copies the workflow at start, so editing never changes a task in flight. Task types still referenced by past runs are archived, not deleted.
6. **Templates** (`render.ts`): `{{input.field}}`, `{{steps.key.output}}`, and later `{{loop.index}}`. Keep it a small, safe substitution, not a general expression language.
7. **Export and import.** A package is the manifest without connection ids or secrets; importing asks which connection fills each role.
8. **Listing.** `list` and `get` return month spend, running and waiting counts, and per-role month statistics; other people's personal workspaces are filtered in SQL.

Done when. `builder.test.ts` shows determinism; saving an invalid workflow is refused with a named step; export then import gives an equal workspace.

STAGE 11 Budgets and the usage ledger IN REPO

Goal. Money is checked before it is spent, at three levels, and every charge is recorded with where the figure came from.

Depends on	Stages 3, 8, 10
Files	`packages/core/src/budget.ts`
Tables	`budget_reservations` (held, settled, released), `usage_ledger`

BUILD STEPS

1. **Pricing.** `pricingFor(models, model)` reads prices a module reports; `costOf(pricing, usage)` charges input and output per million tokens, cache reads at 0.1 of input and cache writes at 1.25.
2. **How much to hold.** `upperBound` is the worst case (whole prompt plus every allowed output token). `historicalP90` is the 90th percentile of this step's past costs in this task type when there is enough history. Hold the p90 when known, the upper bound otherwise.
3. `reserve(q, request)` runs in a transaction, locks the workspace row, and checks all three limits with money already held counted as spent:
   - the task's own limit,
   - the workspace's monthly budget,
   - the connection's monthly spend limit.

   It throws `budget_exceeded` with a sentence that says which limit and what to change.
4. `settle(q, id, actual)` after the call; `release(q, id)` when the call failed or was cancelled. A step must never end with money still held.
5. **Ledger row** per charge: workspace, run, step, connection, role, module, model, tokens, cost, cost source, reservation.
6. **Estimates before a run.** Median and p90 cost and median duration from past runs of the task type, with a confidence level based on how many runs there were and which cost sources they used.

DESIGN NOTES

- Reservation is what makes parallel steps safe: two steps cannot both spend the last dollar.
- The cost source label is cheap to write and is the basis of an honest dashboard.

Done when. Tests show a step refused at each of the three limits, a reservation released on failure, and a ledger row matching the reported usage.

STAGE 12 Orchestrator: runs, steps, approvals IN REPO

Goal. Take a task type and a brief, and carry the run to an end: plan what can run, run it within budget, retry, wait for people, recover from crashes.

Depends on	Stages 2, 4, 8, 10, 11

Files	`packages/core/src/orchestrator.ts`, `dto.ts`, `conclave.ts` (the facade that wires everything), `test-helpers.ts`
Tables	`runs, run_steps, approvals, inbox_items`

THE LOOP

```
start()   validate brief and setup → insert run (copy of the workflow) → engine.wake → emit -
> audit
advance(runId, signals, heartbeat)          // the engine's handler
  load run + step rows
  recoverInterrupted      rows left "running" by a crash become retries
  applySignals            approvals, answers, outside events
  plan                    which steps are runnable, waiting, done
  if nothing runnable     finish, or go idle / reschedule for a retry or deadline
  else                    run every runnable step in parallel, then loop
```

BUILD STEPS

1. **Start.** Validate the brief against the input fields. Check every enabled step has a role with a node, unless the step is optional. Refuse when the month budget is already used. Title the run from the first filled field. Insert, wake, emit, audit, all in one transaction.
2. **Step rows.** One row per attempt: `step_key, kind, attempt, status, role_key, connection_id, node_label, input, output, error, error_code, tokens, cost, cost_source, latency_ms, idempotency_key (unique), retry_at, round`. Statuses: `pending, running, waiting, succeeded, failed, cancelled, superseded, skipped, not_taken`.
3. **Plan.** A step is runnable when every step in its `needs` is finished in a readable state. Independent steps run together; the DAG decides.
4. **Role step.** Check setup, render the prompt, reserve money, mark running, call `model.generate` with the full `NodeCall`, stream deltas to the bus, then in one transaction: write the result, the ledger row, settle the reservation, add to the run's cost and call count, emit.
5. **Failure.** Map the error; if it is retryable and attempts remain, set `retry_at` with backoff (default 3 attempts, 1 s, factor 2) and reschedule. Otherwise fail the step; if the step is optional, skip it; else fail the run and put a "problem" item in the inbox.
6. **Guards.** Per-run call cap (200) and per-node cap (10) stop runaway loops. Step timeouts abort the call through the signal.
7. **Approval step.** Insert an approval and inbox items for people who may decide, mark the step waiting, set the run to waiting. `decide(approve | reject | changes)` writes the decision and sends a signal. "Changes" stores the note as feedback, supersedes the affected steps and re-runs only those.
8. **Cancel.** Mark the run and its open steps cancelled, release held money, abort in-flight calls through their `AbortController`.
9. **Finish.** The run's output is its output step's output. Put a "result" item in the inbox and resolve open items.
10. **Crash recovery.** A row still "running" when the run is next advanced belongs to a dead worker: count it as an interrupted attempt and retry with the same idempotency key.
11. **Facade.** `createConclave({ db, modules, masterKey, … })` migrates, builds every service and returns them, with `startBackground()` (engine worker, outbox relay, notifier, health checks) and `stop()`.
12. **Test harness.** `harness()` gives an in-memory instance with the mock model, a Research Brief workspace, and `settle()` that drains the worker and the relay.

Done when. `orchestrator.test.ts` covers: a run to approval and to the end; retry then success; failure then inbox item; request changes re-running only what is affected; cancel; budget refusal; recovery of an interrupted step; parallel steps.

STAGE 13 Events, outbox and realtime IN REPO

Goal. Every open screen updates by itself, and a dropped connection loses nothing.

Depends on	Stages 3, 12
Libraries	`@fastify/websocket` (server side, Stage 15)
Files	`packages/core/src/events.ts`, `apps/server/src/realtime.ts`, `apps/web/src/lib/realtime.ts`
Tables	`outbox`

BUILD STEPS

1. `emit(q, topics, event)` inserts into `outbox` inside the caller's transaction.
2. `OutboxRelay` polls unpublished rows in id order, publishes each to the in-process `EventBus` with a rising `seq`, and marks it published. `drain()` for tests.
3. `EventBus` keeps the last 2,000 envelopes. `replay(topics, lastSeq)` returns what was missed, or null when the buffer no longer reaches that far.
4. Text deltas are transient: `publishTransient` goes straight to the bus and is never replayed.
5. WebSocket hub: on connect send `hello` with the current seq. A client subscribes to topics with its last seq. The server checks the caller may see each topic, replays what was missed, or answers `refetch`.
6. Web client: one socket, reconnect with backoff, `useTopics([...])`. Each event is applied straight into the TanStack Query cache (`setQueryData`) for runs, steps and inbox, and coarser lists are invalidated with a short debounce.

DESIGN NOTE

The bus is in-process, which is right for one container. For more than one, replace the bus with a shared one (Postgres LISTEN/NOTIFY first, NATS at scale) behind the same class. The outbox does not change.

Done when. A test shows an event is published only if its transaction commits, and a client that reconnects with an old seq gets the missed events in order.

STAGE 14 Inbox and notifications IN REPO

Goal. One place where everything that needs a person arrives, and respectful ways to be told about it.

Depends on	Stages 12, 13
Libraries	`web-push` (browser push with VAPID keys) · `nodemailer` (digest email over SMTP)
Files	`packages/core/src/notifications.ts`, `apps/web/public/sw.js`, `apps/web/src/lib/notify.ts`
Tables	`notification_prefs`, `notification_rules`, `push_subscriptions`, `push_keys`, plus delivery columns on `inbox_items`

BUILD STEPS

1. Inbox kinds: `approval, question, result, problem, budget`. Each item has a title, a body, a **path** through nested runs ("Research brief > Editor > needs approval"), and an action (`approve, answer, open, retry, reconnect`).
2. Per person: desktop notifications, push, email digest (off, daily, weekly, at a local hour), quiet hours, time zone.
3. Per workspace per person: all items, important only, or inbox only.
4. A background loop delivers pending items. Quiet hours hold ordinary items until they end; an approval marked `urgent` (a safety check) still goes through.
5. Push: one VAPID key pair per organisation, private key in the vault; only hosts on an allow-list of push services are contacted; a subscription that keeps failing is removed.
6. The digest groups unread items by workspace and links back with the public URL.

> **Done when.** `notifications.test.ts` covers quiet hours across midnight and time zones, urgent items, per-workspace levels, push to a fake endpoint, and a digest sent once.

STAGE 15 API server IN REPO

> **Goal.** A REST API and a realtime socket where no route can exist without an access rule.

Depends on	Stages 7 to 14
Libraries	Fastify 5 · `@fastify/cookie` · `@fastify/cors` · `@fastify/static` · `@fastify/websocket` · zod 4 · `@node-saml/node-saml` and `@xmldom/xmldom` (SAML)
Files	`apps/server/src/`: `main.ts`, `app.ts`, `config.ts`, `realtime.ts`, `auth/guard.ts`, `auth/oidc.ts`, `auth/saml.ts`, `routes/auth.ts`, `routes/org.ts`, `routes/work.ts`, `routes/notifications.ts`, `routes/hooks.ts`

BUILD STEPS

1. **Config** from the environment with Lite defaults: PGlite in `./data`, bound to localhost, no outside services. When the server listens beyond localhost, generate a setup code and print it; the first owner needs it.
2. **The guard.** Every `/v1` route declares `config.guard`: `{ auth: "public" }` or `{ auth: "user", perm?, workspace? }`. A `preHandler` hook refuses any route without one, and a test fails if one is added without it. Workspace-scoped permissions resolve the workspace from the URL (directly, from a task, or from an approval).
3. **Who is calling.** `Authorization: Bearer cnc_...` for tokens; otherwise the session cookie (HttpOnly, SameSite=Lax, Secure when the public URL is https).
4. **CSRF.** Browser requests that change data must send `x-requested-with: conclave`. A cross-site form cannot set it.
5. **Errors.** One error handler turns `ConclaveError` and zod errors into RFC 9457 problem details (`type, title, status, detail, errors`).
6. **Validation.** zod schemas for every body and query. Let the service do domain validation so messages can name the step or field.
7. **Idempotency.** POSTs that start work accept `Idempotency-Key`; the same key returns the first answer.
8. **Rate limits.** A small in-memory limiter on sign-in, SAML, pairing and callback routes.
9. **Sign-in providers.** OIDC with PKCE and state for Google, GitHub and any issuer (discovery document). SAML with a signed assertion, audience check, one-time request ids within ten minutes, and certificate expiry

warnings.

10. **Static web app.** Serve `apps/web/dist` with a fallback to `index.html` for client routes. CORS only for localhost dev origins.
11. **Health.** `GET /v1/health` for the container health check.
12. **Shutdown.** On SIGINT and SIGTERM: close the app, stop the worker and relay, close the database, with a five-second hard stop.

Appendix X3 lists every route.

> **Done when.** `app.test.ts` walks setup, sign-in, a connection, a workspace, a task and an approval over HTTP; checks 401, 403 and 404 cases; and asserts every registered route has a guard.

STAGE 16 Web app: shell and core screens IN REPO

> **Goal.** A person who has never read the documentation connects a model, creates a workspace, runs a task and approves it.

Depends on	Stage 15
Libraries	React 19 · Vite 8 with `@vitejs/plugin-react` · TanStack Query 5 (server state) · TanStack Router 1 (typed routes) · Tailwind CSS 4 with `@tailwindcss/vite` · `@fontsource-variable/inter` and `@fontsource/jetbrains-mono` (fonts bundled, no outside requests) · `@simplewebauthn/browser`
Files	`apps/web/src/` : `main.tsx` , `styles.css` , `lib/` (api, auth, realtime, theme, format, palette, notify), `ui/` (index, icons), `shell/` (AuthGate, AppShell, Sidebar, CommandPalette), `features/`

BUILD STEPS

1. **Design tokens** in `styles.css` as CSS variables for light and dark (`data-theme` on the root), mapped into Tailwind with `@theme inline` : surfaces, lines, three ink levels, accent, good, warn, bad, nine tile palettes, and five chart series colours. Never hard-code a colour in a component.
2. **UI kit** (`ui/index.tsx`): `Button` (six variants, three sizes), `Pill` , `StatusPill` (one table from status to label and tone), `Tile` , `Card` , `SectionTitle` , `Field` , `Input` , `Textarea` , `Select` , `Segmented` , `ChipGroup` , `ErrorNote` , `Empty` , `Progress` , `PageHeader` .
3. **API client** (`lib/api.ts`): one `request()` that sends the CSRF header, parses problem details into an `ApiError` , and calls a signed-out handler on 401. One typed function per route, using the contract's DTOs.
4. **Auth**: `AuthGate` loads `/v1/auth/state` and shows setup, sign-in, the second step or an invite; inside the app `useMe()` , `useCan(perm)` and `useWorkspaceCan(ws, perm)` decide what to offer. The server still checks every call.
5. **Shell**: a sidebar (search, New task, Home, Inbox with a count, workspaces with status, Connections, Insights, Settings, the person and theme toggle) that becomes a drawer under 768 px; a command palette on Ctrl or Cmd K.
6. **Screens, in this order:**

Screen	What it must do
Home	Onboarding checklist until the first result; "Needs you" with inline approve; "Running now" with progress; workspaces with spend
Connections	Grouped by port; status pill, month spend, used-by; test, turn off, delete; add form built from the manifest, with what the module may reach

New workspace	Guided questions with a live preview, a template, an import, or blank; choose the default node
Workspace · Tasks	Brief form from the task type's fields, an estimate before running, a filterable list, and the task panel
Task panel	Live steps with streamed text, the approval box, the result with copy, the brief, cost against the limit, cancel, and "Open trace"
Workspace · Team	Each role, the node that fills it (connection plus model, tool, skill or workflow), instructions, month statistics
Workspace · Workflows	Task types as cards; opens the node editor (Stage 17)
Workspace · Library, Activity, Settings	Finished results; every task with cost; budgets, members, export
Inbox	Filter by kind and workspace; approve, answer, open; mark read
Settings	Profile, security (password, passkeys, two-factor), notifications, members and invites, access tokens, single sign-on, devices

7. **Accessibility from the start**: real buttons and links, labels on every field, `aria-expanded` and `aria-current`, visible focus, reduced-motion support, no sideways page scroll at phone width, and scrollable regions that take keyboard focus.

> **Done when.** `scripts/smoke.mjs` passes against a running instance, and a new person completes the first task from the Home checklist without help.

PART C

Composition and interoperability

Stages 17 to 28 turn the product into a platform: people compose their own workflows, unlike outside systems join through the same call path, and the whole run can be seen and governed.

STAGE 17 Node editor IN REPO

> **Goal.** Build and change a workflow on a canvas, with a side panel for each step, without writing JSON.

Depends on	Stages 10, 16
Libraries	None added. The canvas is hand-built with SVG and pointer events, which keeps the bundle small and the keyboard behaviour under control
Files	`apps/web/src/features/workspaces/workflows/editor/` : `WorkflowEditor.tsx` , `Canvas.tsx` , `Inspector.tsx` , `AddMenu.tsx` , `Logic.tsx` , `graph.ts`

BUILD STEPS

1. `graph.ts` : pure functions from a workflow definition to nodes and edges and back; auto-layout by depth; add, remove, connect, rename a step and fix every reference to it.
2. Canvas: pan, zoom, drag nodes, drag from a node's dot to another to say "runs after", a dotted grid that follows pan and zoom, sticky notes, coloured connectors with a legend.
3. Keyboard: arrow keys move the selected node; Delete removes; everything dragging can do is also in the inspector's "Runs after" list.
4. Inspector: role and node picker, prompt with insertable variables, brief fields, output format, timeout, retries, optional, turn off.
5. Add menu with search, grouped by kind (model, tool, approval, logic, workspace, agent, automation).
6. Undo and redo as a stack of definitions. Save validates on the server and shows errors on the step they name. A blank workspace opens straight in the editor.

> **Done when.** A workflow built only with the keyboard saves, runs, and reloads with the same layout.

STAGE 18 Tool port and the MCP dock IN REPO

> **Goal.** Any MCP server's tools become nodes a step can call.

Depends on	Stages 5, 12
Libraries	`@modelcontextprotocol/sdk` (the official client; Streamable HTTP with a fallback to SSE)
Files	`modules/mcp` ; the `tool` step in `workflow.ts` ; `runToolStep` in the orchestrator

BUILD STEPS

1. Module config: server URL, optional bearer token, price per call. Egress is the server's host.
2. `listTools` maps MCP tools to `ToolInfo` with their JSON input schema. Tools that write are `sensitive` by default until reviewed.

3. `call` passes arguments, returns structured content, maps MCP errors to contract codes and honours the abort signal.
4. Tool step: a role on `tool@1` whose node is the tool name; arguments are templates. Reserve the price per call, call, charge it as "estimated".
5. Use the official SDK rather than writing the protocol. MCP keeps moving (versions, authorisation, transports); the SDK tracks it.

> **Done when.** The module passes conformance against a fake MCP server, and `tools.test.ts` runs a workflow with a tool step followed by a model step.

STAGE 19 Branching, connectors and loops IN REPO

> **Goal.** Workflows decide, not only proceed.

Depends on	Stages 12, 17
Files	`packages/contract/src/conditions.ts`, the logic step kinds in `workflow.ts`, `graph`, `plan` and `progressLoops` in the orchestrator, `Logic.tsx` in the editor; design in `docs/workflow-logic.md`

BUILD STEPS

1. **Logic steps** (they cost nothing):
   - **If / Switch**: cases made of rules; first match or every match; Else.
   - **AI router** (`classify`): a model picks one of your categories; an answer that names none goes to Other.
   - **Merge**: first input with text, all joined, or a JSON object by step.
   - **Text**: a template, no model.
   - **Stop**: end as done with a result, or as failed with a reason.
2. **Connectors** between steps: Always; Only if (rules on the output or a JSON field); Otherwise; On failure (taken only after retries are used up, and then the failure counts as handled). A connector can carry an instruction for the next step.
3. **Rules** (`testCondition`): contains, equals, starts with, matches, is empty, number comparisons, on the text or a JSON path. Pure functions shared by server and editor.
4. **Deciding** (`connectorTaken`): each connector is undecided, taken or not taken. A step whose incoming connectors are all not taken becomes `not_taken`, and that flows downstream. Where branches meet, a step waits for all, any, or the first.
5. **Loops.** A loop connector sends work back to an earlier step until rules hold, up to a round limit (1 to 20, default 3). Each round supersedes the last round's rows, and `{{loop.index}}` and the previous output are available. At the limit, carry on with the last round or fail, as chosen.

> **Done when.** `logic.test.ts` covers every connector kind, joins, a router picking each output, and loops ending by rule and by limit. Logic steps write no ledger rows.

STAGE 20 Workspace steps (sub-tasks) IN REPO

> **Goal.** A workflow can run another workspace's task type, or its own, and carry on with the result. This is what makes workspaces composable.

Depends on	Stages 12, 19
Files	`startCall`, `progressCalls`, `childFinished`, `cancelTree`, `path` in the orchestrator; `truncateWorkflow` in the contract
Columns	`runs.parent_run_id, parent_step_id, root_run_id, depth, iteration`

BUILD STEPS

1. The step names a workspace (or "self"), a task type, and inputs as templates. Optional "run up to this step".
2. **Repeat modes**: a fixed number of times; once for each item of a list; until the output contains some text.
3. **Permission.** The sub-task runs as the person who started the parent, and only if that person may run tasks in the target workspace.
4. **Budget.** Each sub-task gets a share of what is left of the parent's limit (after cost, holds and money already committed to siblings). Its cost is added to the parent's step and run when it finishes.
5. **Guards.** At most 5 levels deep; at most 100 sub-tasks per task. A workflow may call itself only if the inner run stops before reaching that call; the editor refuses one that never would.
6. **Lifecycle.** A child finishing wakes its parent. Cancelling a task cancels everything under it. A child's approval reaches the inbox with the full path, and the parent reports once, at the end.

> **Done when.** `nesting.test.ts` covers cross-workspace calls, "run up to", each repeat mode, recursion and its guard, depth and count caps, budget shares, cancel of a tree, and approval paths.

STAGE 21 Agent and Workflow ports IN REPO

> **Goal.** Hand work to an outside agent or workflow engine and wait as long as it takes, across restarts, without holding a worker.

Depends on	Stages 2, 4, 12
Files	`AgentPortV1`, `WorkflowPortV1` in `ports.ts`; `agent` and `automation` steps; `startExternal`, `watch`, `progressExternal`, `applyExternal`, `completeExternal`, `failExternal`, `openQuestion`, `answerAgent`, `deliverCallback` in the orchestrator; `routes/hooks.ts`; `QuestionPanel.tsx`; design in `docs/agent-workflow-ports.md`
Tables	`external_callbacks` (token hash, run, step, used)

BUILD STEPS

1. **Start.** Reserve the step's cap, or the connection's prices (per call plus per minute) over the whole timeout. Call `startRun` or `trigger`. Write the handle or execution into the step's `input` as the checkpoint. Mark the step `waiting` and return; the worker is free.
2. **Agents.** A watcher in the worker process reads `streamEvents`. Progress lines are written to the step (last 20 kept, flushed at most once a second). A question, the result or a failure goes through an engine **signal**, so the run's own worker applies it under its lease.
3. **Questions.** `needs_input` opens one inbox item of kind "question" with the full path. The answer is sent with `provideInput` and the watch resumes. The same question seen again is not asked twice. A step may instead be set to fail on questions.
4. **Workflows.** If `trigger` returns "running", poll with backoff from 2 s to once a minute, or wait for a callback. The callback URL carries a 256-bit token; only its hash is stored and it works once.
5. **Restarts.** Waiting steps are re-checked every 30 s. A worker that restarted picks the watch up again; the port contract requires `streamEvents` to work on a run already under way and to yield a finished run's final event

again.

6. **Cost.** Charge what the system reports ("reported"), else the connection's prices for the time taken ("estimated").
7. **Ending.** A timeout asks the outside system to stop and frees the held money. Cancel does the same; where a system cannot cancel, its result is ignored. Outside runs are not retried unless the step asks, because they are not idempotent.
8. **UI.** The task panel shows "working outside" with the latest progress lines, the outside id, and the question box. The Team tab picks a skill or a workflow as the node.

> **Done when.** `external.test.ts` covers: an agent run to the end; a question and its answer; fail-on-question; a restart in the middle; timeout with release; cancel; On failure catching it; polling; a callback that works once.

STAGE 22 Module protocol and the docks IN REPO

> **Goal.** The same module code runs inside the server, on its author's server, as a process the server starts, or on someone's computer. Only the transport and the security boundary differ.

Depends on	Stages 5, 8, 21
Libraries	`node:http`, `node:child_process`, `node:readline`, `node:crypto`. No dependency added
Files	`packages/module-sdk/src/cmp.ts`; `modules/remote-dock`, `modules/sidecar-dock`, `modules/webhook-workflow`, `modules/sample-agent`; `apps/server/src/docks.test.ts`

THE CONCLAVE MODULE PROTOCOL (CMP 1.0)

CMP is the contract between Conclave and its modules. It is JSON-RPC 2.0, which is a message format and not the point. Do not position it against MCP or A2A; those are adapters that sit behind ports.

Direction	Methods
Host to module	`initialize` (answers with the manifest), `health`, `outsideVersion`, `discover`, `model.listModels`, `model.generate`, `tool.listTools`, `tool.call`, `agent.listSkills`, `agent.startRun`, `agent.events`, `agent.provideInput`, `agent.cancel`, `workflow.list`, `workflow.trigger`, `workflow.poll`, `workflow.callback`, `workflow.cancel`, and the `$/cancel` notification
Module to host (stdio)	`host.fetch` (network calls pass the host's allow-list), `host.log`

BUILD STEPS

1. **Dispatcher.** `createDispatcher(module)` turns method names into port calls, tracks in-flight calls for `$/cancel`, and encodes errors with the contract code and `retryable`.
2. **Agent events over request and response.** `agent.events` is a long poll over a cursor (up to 20 s), so one method works over HTTP and stdio.
3. **Serving.** `serveHttp(module, { secret })` and `serveStdio(module)` serve any `defineModule` module unchanged.
4. **HTTP signing.** Every request carries a timestamp, a nonce and `v1=HMAC-SHA256(secret, "ts.nonce.body")`. Refuse requests older than five minutes, a nonce seen in the last ten, and secrets shorter than 32 characters. Sign every answer over the request's nonce. Plain http only for loopback.
5. **Host side.** `httpClient`, `spawnClient` and `cmpInstance` give core an ordinary `ModuleInstance`. The version is checked at `initialize`.

6. **Remote dock** (`conclave/remote`): URL, shared secret, prices.
7. **Sidecar dock** (`conclave/sidecar`): a command the server starts without a shell and with a bare environment (no keys, no database URL). Only commands listed in `CONCLAVE_SIDECARS` may start. The module's `fetch` goes through the host. For real isolation make the command a container with no network.
8. **Docked modules.** Testing a docked connection asks its module for its manifest, stores the ports it provides (so role pickers offer only fitting connections), and says so when a later test finds a different module, version or wider egress.
9. **Webhook workflow dock** (`conclave/webhook-workflow`): POST the inputs as JSON; in "wait" mode the response is the result, in "callback" mode the workflow posts to the callback URL. With a secret, sign the request with HMAC. This already works with n8n, Zapier, Make, Activepieces and Windmill, with no module code.
10. **Sample agent**: a module made only with the public SDK (two skills, two workflows) to use as a template and as the test target for every dock.

> **Trust.** A signature shows where a package came from and that it was not changed. It does not show the module is safe. The host must still enforce permissions, egress, resource limits and secret isolation for every dock.

> **Done when.** `docks.test.ts` runs the sample agent through the remote and sidecar docks end to end, and shows a bad signature, a replayed nonce and an unlisted sidecar command are refused.

STAGE 23 Local Bridge IN REPO

> **Goal.** A server in the cloud can use a module or an app on a person's own computer, without that computer opening a port, and only for things its owner added there.

Depends on	Stage 22
Libraries	`node:crypto` (Ed25519), the platform WebSocket. No dependency added
Files	`apps/bridge/src/index.ts` , `cli.ts` ; `packages/module-sdk/src/bridge-protocol.ts` ; `apps/server/src/bridge.ts` ; `packages/core/src/devices.ts` ; `DevicesSection.tsx`
Tables	`bridge_devices` , `bridge_pairings`

BUILD STEPS

1. **CLI.** `conclave-bridge pair <server-url>` , `target add <name> --stdio "<command>" [--egress hosts]` , `target add <name> --http <loopback url>` , `target list` , `target remove` , `run` , `status` .
2. **Pairing.** The device makes an Ed25519 key pair and shows a one-time code (8 characters, 10 minutes, stored hashed). A signed-in person enters it in Settings, Devices. The device shows who claimed it and asks its owner to confirm. The person is told in their inbox.
3. **Connection.** One outbound WebSocket per device. The device proves itself by signing the server's random challenge.
4. **Signed calls.** Every call the server sends is signed with the server's Ed25519 key, derived from the master key and pinned by the device at pairing. The bridge checks the signature, a five-minute window, that the call was not seen before, that the method is a host-to-module CMP call, and that the target exists on that computer.
5. **Default deny.** Nothing is reachable until the owner adds a target at the computer. The server cannot add or widen one. Targets are read from disk on every call, so removing one works at once.

6. **Bridge dock** (`conclave/bridge`): a connection names a device and a target. Removing a device disconnects it at once.
7. **Local activity log.** Time, target and method for every call; never contents.

Still to build, with the desktop app: the device key in the OS keychain; OS sandboxing of targets; local prompts for sensitive actions; per-target rate and size limits; a daily activity summary; a tray pause.

> **Done when.** Tests pair a device, run the sample agent through it, and show an unsigned call, an unknown target and a removed device are all refused.

STAGE 24 A2A dock IN REPO

> **Goal.** Any agent that speaks Agent2Agent becomes an Agent-port node, with no module code per agent.

Depends on	Stages 5, 21
Libraries	None. JSON-RPC over `ctx.fetch`
Files	`modules/a2a/src/index.ts` , `index.test.ts`

BUILD STEPS

1. **Config.** Agent address (base URL or the card's URL), optional bearer token, extra hosts, seconds between checks, prices. Egress is `{config.url}` plus `{config.extraHosts[]}` , because a card may send calls to another host.
2. **Agent Card.** Read `/.well-known/agent-card.json` , then the older `/.well-known/agent.json` . Cache it for five minutes.
3. **Pick the version from the card.**

	Protocol 1.0	Protocol 0.3
Endpoint	`supportedInterfaces[]` with `protocolBinding: "JSONRPC"`	top-level `url`
Methods	`SendMessage, GetTask, CancelTask`	`message/send, tasks/get, tasks/cancel`
States	`TASK_STATE_COMPLETED` ...	`completed` , `input-required` ...
Role, parts	`ROLE_USER` ; `{ text }` , `{ data }`	`user` ; `{ kind: "text", text }`
Do not wait	`configuration.returnImmediately: true`	`configuration.blocking: false`
Result	`{ task }` or `{ message }`	bare, marked by `kind`
Header	`A2A-Version: 1.0`	none

4. **Port mapping.** `listSkills` from the card. `startRun` sends a message and keeps the task id and context id as the handle. `streamEvents` asks for the task again until it ends, yielding progress when the status message changes. `provideInput` sends a new message with the same task and context id. `cancel` calls CancelTask and treats "already ended" as success.
5. **States to events.** completed gives the artifacts' text and data; `input-required` gives one question; failed, rejected, canceled and `auth-required` become contract errors, so retries and On failure behave as for any node.

6. **Trace.** Put the step's `traceparent` in the message metadata.
7. **Cost.** Read `metadata.usage` on the finished task when present; else the connection's prices.
8. An agent that answers with a message and no task finishes at once.

Polling is used on purpose: it needs nothing beyond the base protocol and survives a restart of either side. Streaming (`SendStreamingMessage`) and push notifications would only make progress arrive sooner; add them later.

> **Not yet proven.** The module passes the conformance kit against a fake agent for both versions. Run it against one real A2A agent before relying on it.

> **Done when.** Conformance passes for 1.0 and 0.3, with and without a question, and the specific tests check headers, method names and reported usage.

STAGE 25 Installed modules IN REPO

> **Goal.** A developer adds a module for a system the Conclave team has never heard of, without changing Conclave, and it is at once a node in existing workflows with budgets, traces and audit working.

Depends on	Stages 5, 8, 15
Files	`apps/server/src/modules.ts`, `modules.test.ts`; `ModuleRegistry.install(module, source)`; `ModulePermissions` in `ConnectionsPage.tsx`

BUILD STEPS

1. At start, load modules from `<data dir>/modules` (`CONCLAVE_MODULES_DIR`): every `.js`, `.mjs` or `.ts` file, and every sub-folder with a `package.json` entry or an `index` file. Also load each name or path in `CONCLAVE_MODULES`.
2. A module's default export is `{ manifest, create }`. Validate and normalise the manifest with the same schema as a built-in module.
3. A broken module is skipped and the reason logged. One bad file must not stop the server.
4. An installed module may not take the id of a module already present.
5. Mark it "installed" in the registry. In the connection form show "Added by your admin", which hosts it may reach and which secrets it reads, before anything is saved.

> **Trust.** These modules run inside the server with its privileges. Installing one is the server admin's decision. Code that should not be trusted that far belongs in the remote, sidecar or bridge dock, where a process boundary does the isolating.

> **Done when.** A test writes a module file to a temporary folder, loads it, calls its tool, and shows that bad manifests, a throwing file and an id clash are each left out with a reason.

STAGE 26 Trace IN REPO

> **Goal.** One coherent history for a task that crosses systems and workspaces, instead of several unrelated logs.

Depends on	Stages 11, 12, 20, 21

Files `packages/core/src/insights.ts` (`trace`), `traceparent` in `contract/call.ts`, `GET /v1/tasks/:id/trace`, `apps/web/src/features/workspaces/tasks/TracePage.tsx`

BUILD STEPS

1. **One trace id for the tree.** Every call uses the top task's id as its W3C trace id, and `parentCallId` is the workspace step that started the task.
2. **Read the tree.** From any task, find its root, then load every run with that root, every step attempt, reservations summed per step, and decisions with the decider's name. Four queries, no recursion.
3. **Spans.** A run is a span; each step attempt is a span under its run; a sub-task hangs under the step that started it. Retries and loop rounds are separate spans.
4. **Per span.** Kind of system (model, tool, agent, workflow engine, person, sub-task, logic), node label, connection and module, status, start and end, cost and its source, tokens, money reserved and still held, the decision, the error and its code, the outside id, the `traceparent` that was sent.
5. **No double counting.** A workspace step's cost is its sub-tasks' cost, so its own share is zero. Per-system totals must add up to the task's total.
6. **Access.** The caller must be able to see the task they ask about. Spans in someone else's personal workspace stay in the tree, with names, labels, errors and notes left out.
7. **Screen.** Summary tiles (duration, cost of limit, calls, failures, decisions); "Where the time went" as one stacked bar by kind of system; "Systems it reached" with calls, cost and time; the timeline as a waterfall with one row per span, indented by nesting; a row opens to its details. Refresh every 2.5 s while the task is live.
8. **Chart rules.** Five series colours in a fixed order, validated for colour-blind separation in light and dark; every coloured mark has a text label beside it; tasks and logic are neutral grey.

The trace is read from the same rows the budget uses, so it cannot disagree with the ledger, and the Lite profile stays Postgres-only. The ids are standard, so OpenTelemetry export later is a relay, not a redesign.

> **Done when.** `insights.test.ts` shows a parent and a sub-task in another workspace in one trace from either end, unique span ids, system totals equal to the task total, and a refusal across organisations.

STAGE 27 Insights and the audit trail IN REPO

> **Goal.** The control plane's view of itself: what every connected system did and cost across all workspaces, and who changed what.

Depends on Stages 7, 11, 26

Files `insights.ts` (`usage`, `audit`), `GET /v1/insights/usage`, `GET /v1/audit`, `apps/web/src/features/insights/InsightsPage.tsx`

BUILD STEPS

1. **Usage** over 7, 30 or 90 days, limited to workspaces the caller can see: totals; the period before for comparison; one entry per day including empty days (group by a UTC date string, not a timestamp, so both database drivers agree); top twelve by workspace, by connection and by model; the split by cost source; tasks done and failed.
2. **Reliability** per connection from step attempts: calls, failures, retries, average time. Count every attempt, so a connection that needed retries shows even when the task ended well.
3. **Audit**: newest first; the actor's name; the target's name looked up across runs, workspaces, connections, approvals and members; a filter by area (the part of the action before the dot); paging by id. Owners only,

because it names people.

4. **Screen.** Four tiles (spent with change, calls, tasks with failures, how exact the costs are); spend per day as bars with hover and a table view; three ranked lists with proportional bars; cost sources with plain explanations; a reliability table; and an Audit tab with plain-language sentences ("Ada approved Tidal energy").
5. Make the sidebar's month figure the way into this page.

> **Done when.** Tests check the sums, that every day is present, that other people's personal workspaces are left out, and audit paging and filtering.

STAGE 28 Tests, CI, accessibility, packaging IN REPO

> **Goal.** Every change is checked the same way on every machine, and one command starts a real instance.

Libraries	Vitest 5 · Playwright 1.56 · `@axe-core/playwright` · Docker · GitHub Actions · test-only: `selfsigned`, `xml-crypto`
Files	`e2e/server.ts`, `e2e/seed.setup.ts`, `e2e/a11y.spec.ts`, `playwright.config.ts`, `scripts/smoke.mjs`, `scripts/test-pg.mjs`, `Dockerfile`, `compose.yaml`, `compose.test.yaml`, `.github/workflows/ci.yml`

BUILD STEPS

1. **Unit and service tests** next to the code, on in-memory PGlite, with the mock model and fake `fetch`. No test touches the network.
2. **The same suite on Postgres**: `compose.test.yaml` starts a throwaway database; `npm run test:pg` gives each test its own schema.
3. **Smoke test**: `scripts/smoke.mjs` sets up an owner, runs a mock Research Brief and approves it against any running instance.
4. **Accessibility**: a seeded throwaway server; open every screen and state; run axe at WCAG 2.2 AA in light, dark and at phone width; assert the page never scrolls sideways. Add each new screen to the list in `a11y.spec.ts` when you build it.
5. **CI jobs**: (1) typecheck, name lint, tests, web build; (2) the suite on Postgres, then boot the server and run the smoke test; (3) accessibility; (4) `npm audit` at high severity.
6. **Packaging**: a two-stage Dockerfile on Node 24 slim, running as the `node` user, with `/data` as a volume and a health check; Compose pairs it with `pgvector/pgvector:pg17` for the Lite profile.

> **Done when.** A clean clone passes all four CI jobs, and `docker compose up -d` gives a working instance.

PART D

What to build next

Everything here is a recommendation from the research, in the order I would build it. None of it exists yet.

STAGE 29 Governance across nested calls IN REPO

> **Goal.** A nested call can never do more than the call that started it. Budgets already narrowed; now permissions do too.

> **As built.** Simpler than the plan below, on purpose. Files: `packages/core/src/policies.ts` (`decide` , `Policies`), `gate` , `policyFor` , `approvedBefore` in the orchestrator, migration `0008_policies` , `PoliciesCard.tsx` , and the "Rules" line in the trace.
>
> - A rule is one connection, or one tool on it, set to **allow**, **ask** or **deny**, for a workspace or for the whole organisation. No general subject/action/resource language yet.
> - Sources are the organisation, every workspace above the task (copied onto the sub-task as `runs.inherited` when it starts; only limits travel, never "allow"), and the task's own workspace. The strictest wins. Within one source a tool rule beats a connection rule.
> - The default is allow, not deny, because assigning a node to a role is already a grant. The exception: a tool marked `sensitive` defaults to ask until a rule names that tool.
> - "Ask" is satisfied by an approval step that leads to this step and was approved. It does not open an approval by itself.
> - The check runs before `reserve` . A refusal fails the step with the new code `forbidden` ; the decision is kept on `run_steps.policy` .
> - Still to build from the plan: gateway keys for outside agents (step 9), the editor's warning (step 6), and a check at task start.

The original plan, kept for the parts not yet built:

BUILD STEPS

1. **Policy as data.** New table `policies (id, org_id, workspace_id null, effect allow|deny, subject, action, resource, conditions jsonb)` . Subjects: a role, a workspace, a connection. Actions: `call:node` , `call:tool` , `start:task` , `use:sensitive` .
2. **One decision function.** `decide(policySet, call) → { allowed, reason, policyId }` , pure and unit-tested. Deny wins; no match means deny.
3. **Grant at the top.** When a task starts, compute the set of permission strings its workflow needs and store it on the run.
4. **Narrow on the way down.** A sub-task's grant is the intersection of its parent's grant and what its own workspace allows. Put the result in `NodeCall.permissions` .
5. **Check before reserve.** In each step, call `decide` before `reserve` . A denial fails the step with a new code `forbidden` (add it to the error codes; not retryable) and a sentence naming the policy.
6. **Sensitive tools.** A tool marked `sensitive` needs either a policy that allows it or an approval step before it. The editor warns when neither is there.

7. **Record the decision** on the step row, and show it in the trace's detail panel: "Allowed by policy X" or "Refused: Y".
8. **UI.** Workspace, Settings, "What this workspace may use": a table of connections and tools with allow, ask first, or never. Keep it to those three words.
9. **Gateway keys for outside agents.** When an agent calls back into Conclave (through MCP or the API), give it a short-lived key bound to the run, carrying the run's grant and remaining budget, so its calls are children of the step that started it.

> **Done when.** A test builds a three-level nest where level three tries a tool level one was never allowed. It is refused, the refusal is in the trace, and no money was held.

STAGE 30 The remaining roadmap, in order PARTLY BUILT

Rows 2 to 8 are now in the repository; what was built is noted in each.

#	Build	What to do	Why now
1	Prove A2A on a real agent	Connect one public A2A agent of each protocol version; fix what the fake agent missed; then add streaming	Interoperability gate
2	n8n module	IN REPO `modules/n8n`. n8n's API has no call that starts a workflow, so the module lists published workflows over the API, starts one through its Webhook node, finds the execution it started, follows it and stops it on cancel. Tested against a fake n8n only	Listing and cancel are what the webhook cannot do
3	OpenTelemetry export	IN REPO `packages/core/src/otel.ts`. With `CONCLAVE_OTLP_ENDPOINT` set, each finished top-level task is sent once as OTLP/HTTP JSON, with GenAI attributes. Not yet tried against a real collector	The ids are already standard
4	Loop guards	IN REPO for two of three: a node that already answered exactly this input three times in the task tree, and a sub-task that would start a task above it with the same brief. Near-identical (fuzzy) inputs and trigger chains are still to build	Needed before agents call agents
5	Module install from the app	IN REPO `apps/server/src/uploads.ts`, `module-runner.ts`, `routes/modules.ts`, `InstallModule.tsx`. One JavaScript file; its manifest is read in a separate process and shown before anything is installed; each connection runs it as its own process with a bare environment and egress through the server; no restart. Off unless `CONCLAVE_MODULE_UPLOADS=on`. It is a process boundary, not a sandbox	Extensibility gate without shell access
6	Remote dock: short-lived credentials	IN REPO in `cmp.ts` and `modules/remote-dock`. The server signs each request with its own Ed25519 key: good for five minutes, once, for one module address. The module holds only the server's public key and signs answers with its own, which can be pinned. Shared secrets remain, and a module accepts two secrets or several server keys at once for changes without a gap. Mutual TLS is not built	A leaked secret should not last

7	Three real templates	**IN REPO** Support Triage (router and branches), Field Research (an outside agent), Content Desk (one sub-task per item). Not yet published as packages	Shows composition, not features
8	Knowledge port	**IN REPO** `KnowledgePortV1` (`listSources` , `search`) with passages that carry their source and a 0 to 1 score; a **Look up** step run by the same code as tool steps; the port in CMP, the docks and the conformance kit. Two modules: `modules/notes` (a folder of Markdown and text, BM25, inside `CONCLAVE_KNOWLEDGE_DIR` only) and `modules/search-endpoint` (any retrieval endpoint). Changed from the plan: no embeddings module, because a module has no store of its own to keep vectors in; a vector store is reached through the search endpoint dock instead. Core still chunks, embeds and indexes nothing	First port beyond the four; only when a module needs it
9	Trigger port	`trigger@1` for schedules and inbound webhooks that start tasks; a run records its trigger chain	Automation without a person
10	The Hive as an execution explorer	Start from the trace: a graph of workspaces, roles and connections where edge weight is real calls and cost over a period, and a click opens the traces behind it	A graph of real execution, not a picture of entities
11	Standard profile	More than one server container: a shared event bus (Postgres LISTEN/NOTIFY, then NATS), a shared idempotency store and rate limiter (Valkey), sticky-free WebSockets	Only when one container is not enough
12	Second engine	A Temporal implementation of `ExecutionEngine` , behind the same interface, run against the same orchestrator tests	Only when run volume needs it
13	Desktop app	Wrap the bridge CLI: keychain key, tray, local prompts, sandboxed targets. It must not block the web platform	After the bridge's protections
14	Second SDK, marketplace, no-code module builder	Publish CMP as a short specification first; a Python SDK when a Python author asks; a public catalogue when there are modules to list	Only after outside authors exist

Keep deferring the Device, Channel, Media, Storage, Identity and Secrets ports, Kubernetes and multi-region, until a concrete integration or deployment needs each one.

D3 · UI improvements still needed

Where	Improvement	Why
Workflow canvas	Run view: colour each node by its status in a chosen past task, with cost and time on the node, and a link to its trace row	Debug where you build
Workflow canvas	"Try this step" with pinned sample input, without running the whole task	Faster iteration, no spend on earlier steps
Trace	Filter by system or status; jump to the first failure; compare two runs of the same task type side by side	Long traces need narrowing
Trace	Show the step's output and input in the detail panel, loaded on demand	Today you go back to the task for the text

Trace and Insights	Live updates through the realtime topics instead of a 2.5 s refetch	Less load, instant
Insights	Budget forecast: at this rate the month ends at $X of $Y, per workspace	Acts before the limit is hit
Insights	Export usage and audit as CSV; a date-range picker	Finance and compliance ask for files
Insights	Click a connection to see its failing steps with links to traces	From "it fails" to "why"
Connections	A module catalogue page: installed and built-in modules with ports, docks, permissions and versions, separate from the add form	The add form is the only place modules show today
Connections	Show the permission summary on each existing connection card, and warn when a docked module widens its egress	Governance in view, not only at creation
Connections	Group the add form by what you want ("a model", "an agent", "a workflow engine", "a tool", "my own module") with one recommended choice each	The module list is growing
Home	After the first result, replace onboarding with three suggested next steps: add a tool, add an agent, call another workspace	Leads people to composition
Team	Show each role's last failures and average cost per call beside the node picker, and suggest a cheaper node when one fits	Choosing a node is a cost decision
Task panel	A small inline timeline bar per step, so the trace is not needed for a simple task	Most tasks are simple
Inbox	Bulk approve for items of the same task type; keyboard shortcuts for approve and next	Approvals pile up
Everywhere	Tiny amounts show as "$0.0000". Show "under $0.0001" instead	Looks like a bug
Everywhere	Code-split the editor, the trace and Insights; the bundle is over 500 kB	First load on a phone
Everywhere	A manual WCAG 2.2 AA audit with a screen reader before general release	Automated checks find about half

D4 · Gates: how to know it works

ENGINEERING GATES

Gate	What must be shown	State
Interoperability	Three unlike outside systems work through the same call path	Shown with fakes; needs a real A2A agent
Extensibility	A developer builds and installs a module without changing core	Shown by Stage 25's test
Durability	A nested workflow survives a server restart and resumes	Shown by tests
Governance	A nested call cannot exceed inherited permissions or budget	Shown by `policies.test.ts` and `nesting.test.ts`
Observability	One trace rebuilds the whole cross-system run	Shown by Stage 26's test

PRODUCT GATES

1. A new person connects a system and finishes a useful task without reading the documentation.
2. At least three independent people or teams build workflows the founders did not design for them.
3. At least two keep using it after the first experiments, ideally paying or contributing.

These say more than counts of integrations or node types. The risk to avoid is spending months on infrastructure for an ecosystem that has not yet shown demand, so reach the product gates before Stage 30's items 11 to 14.

APPENDICES

Reference

X1 · Every library and why

Library	Used in	Why this one
TypeScript 6, tsx 4	everything	Strict types across server, web and modules; run source with no build step
zod 4	contract, module-sdk, core, server	One schema gives both validation and the type
uuidv7	contract	Ids that sort by time: ordering, index locality, id paging
pg 8	db	Plain SQL with explicit locks
@electric-sql/pglite	db	Real Postgres with no server, for Lite and tests
Fastify 5 with cookie, cors, static, websocket	server	Fast, typed, per-route config for the guard
@node-saml/node-saml, @xmldom/xmldom	server	SAML sign-in; never write XML signature checks by hand
@simplewebauthn/server and /browser	core, web	Passkeys
qrcode	core	TOTP setup code
web-push	core	Browser push with VAPID
nodemailer	core	Digest email over SMTP
@anthropic-ai/sdk	modules/anthropic	Official client: streaming, retries, new features
@modelcontextprotocol/sdk	modules/mcp	Official client tracks a moving protocol
React 19, Vite 8	web	UI and build
TanStack Query 5	web	Server state cache that realtime events write into
TanStack Router 1	web	Typed nested routes
Tailwind CSS 4	web	Utilities bound to CSS-variable tokens
@fontsource inter, jetbrains-mono	web	Fonts bundled; a self-hosted instance makes no outside requests
Vitest 5	tests	Fast, TypeScript-native
Playwright 1.56, @axe-core/playwright	e2e	Accessibility on every screen in three setups
node:crypto	core, module-sdk, bridge	scrypt, AES-256-GCM, HMAC-SHA256, Ed25519, with no dependency

Deliberately not used: an ORM, a queue service, Redis in the Lite profile, a canvas library, a component library, a charting library, a tracing backend.

X2 · Database tables

Migration	Tables and columns added
0001 foundation	`orgs, users, memberships, data_keys, secrets, connections, workspaces, workspace_versions, roles, task_types, runs, run_steps, approvals, inbox_items, budget_reservations, usage_ledger, outbox, engine_tasks, engine_signals, audit_log`
0002 accounts	`identities, sessions, api_tokens, invites, workspace_members`; password and sign-in columns on `users`; `created_by` and `private` on `workspaces`
0003 notifications	`notification_prefs, notification_rules, push_subscriptions, push_keys`; urgency and delivery columns on `inbox_items`
0004 second factors	`user_totp, recovery_codes, passkeys, auth_challenges`
0005 nested runs	`runs.parent_step_id, iteration`; `task_types.archived_at`
0006 loops	`runs.loops`; `run_steps.round`
0007 outside systems	`external_callbacks, bridge_devices, bridge_pairings`; `connections.ports, remote_manifest`
0008 policies	`policies`; `runs.inherited`, `runs.trace_exported_at`; `run_steps.policy`

Indexes that matter: runs by workspace and time, by root, by parent step; live steps per run; held reservations per workspace; ledger by workspace and time and by connection and time; unpublished outbox rows; due engine tasks; pending signals.

X3 · API routes

Area	Routes (all under `/v1`)
Sign-in	`GET auth/state` · `POST auth/setup` · `POST auth/sign-in` · `POST auth/mfa` · `POST auth/passkey/options` · `POST auth/passkey` · `POST auth/sign-out` · `GET auth/invites/:token` · `POST auth/invites/:token/accept` · `GET auth/oidc/:provider/start` and `/callback` · `GET auth/saml/metadata` and `/start` · `POST auth/switch-org`
Me	`GET, PATCH me` · `POST me/password` · `GET me/security` · `POST me/totp/setup, enable, disable` · `POST me/recovery-codes` · `POST me/passkeys/options` · `POST, PATCH, DELETE me/passkeys` · `GET, PATCH me/notifications` · `PUT me/notifications/workspaces/:id` · `POST, DELETE me/push-subscriptions` · `POST me/notifications/test`
Organisation	`POST orgs` · `GET org/members` · `PATCH, DELETE org/members/:userId` · `GET, POST org/invites` · `DELETE org/invites/:id` · `GET org/sso` · `GET, POST tokens` · `DELETE tokens/:id`
Catalogue	`GET modules` · `GET templates` · `GET call-targets`
Connections	`GET, POST connections` · `GET, PATCH, DELETE connections/:id` · `POST connections/:id/test, disable, enable` · `GET connections/:id/models, tools, skills, workflows, sources`

Workspaces	`GET, POST workspaces` · `POST workspaces/compose` · `GET workspaces/:id` · `PATCH workspaces/:id/budget` · `GET workspaces/:id/export` · `POST, PATCH, DELETE workspaces/:id/roles` · `PUT workspaces/:id/roles/:key` · `POST, PUT, DELETE workspaces/:id/task-types` · `GET workspaces/:id/members` · `PUT workspaces/:id/members/:userId`
Tasks	`GET, POST workspaces/:id/tasks` · `POST workspaces/:id/estimate` · `GET tasks` · `GET tasks/:id` · `GET tasks/:id/trace` · `POST tasks/:id/cancel` · `POST approvals/:id`
Inbox	`GET inbox` · `POST inbox/:id/read`
Insights	`GET insights/usage?days=` · `GET audit?area=&before=&limit=`
Modules from the app	`GET modules/uploads` · `POST modules/inspect` · `POST modules/install` · `DELETE modules/uploads/:publisher/:name`
Rules	`GET, PUT workspaces/:id/policies` · `GET, PUT org/policies`
Bridge and hooks	`POST bridge/pairings` · `GET bridge/pairings/:id` · `POST bridge/pairings/:id/confirm` · `POST bridge/claim` · `GET bridge/connect` (WebSocket) · `GET devices` · `DELETE devices/:id` · the one-time callback route for outside workflows
Other	`GET health` · the realtime WebSocket

X4 · Settings

Variable	Meaning
`CONCLAVE_DATABASE_URL`	`pglite:<dir>`, `pglite:memory` or `postgres://`. Default: PGlite in the data folder
`CONCLAVE_DATA_DIR`	Where PGlite, the master key and installed modules live
`CONCLAVE_MASTER_KEY`	32 bytes, base64. Back it up; without it, saved secrets cannot be read
`CONCLAVE_PUBLIC_URL`	The address people use; links and callbacks are built from it; https turns on Secure cookies
`CONCLAVE_SETUP_CODE`	Needed to create the owner on a server that listens beyond localhost
`HOST`, `PORT`	Listen address. Default `127.0.0.1:8787`
`CONCLAVE_GOOGLE_*`, `CONCLAVE_GITHUB_*`, `CONCLAVE_OIDC_*`	OpenID Connect sign-in
`CONCLAVE_SAML_*`	SAML sign-in: metadata, name, certificates, entity id, options
`CONCLAVE_SIGNUP_DOMAINS`	Email domains whose verified users may join as viewers
`CONCLAVE_SMTP_URL`, `CONCLAVE_MAIL_FROM`	Email digests
`CONCLAVE_VAPID_SUBJECT`, `CONCLAVE_PUSH_HOSTS`	Web push contact and allowed push services
`CONCLAVE_WEBAUTHN_ORIGINS`	Extra origins passkeys may be used from
`CONCLAVE_SIDECARS`	Commands sidecar connections may start, separated by `;`, or `*`. Off by default

`CONCLAVE_KNOWLEDGE_DIR`	The one folder Notes folder connections may read. Default `<data dir>/knowledge`
`CONCLAVE_MODULE_UPLOADS`	`on` lets owners install modules from the app. Off by default
`CONCLAVE_MODULES_DIR`, `CONCLAVE_MODULES`	Folder and list of modules installed at start
`CONCLAVE_OTLP_ENDPOINT`, `CONCLAVE_OTLP_HEADERS`	Send finished traces to an OpenTelemetry collector
`CONCLAVE_TEST_DATABASE_URL`	Run the test suite on a real Postgres

X5 · Build checklist

☐ 1 Monorepo, strict TypeScript, Vitest, name lint
☐ 2 Contract: call envelope, errors, ports, workflow schema, events, API types
☐ 3 Database interface, PGlite and Postgres, migrations
☐ 4 Engine: lease, heartbeat, generation, signals
☐ 5 Module SDK: manifest, defineModule, conformance kit
☐ 6 Secret vault: envelope encryption, shred
☐ 7 Accounts, four roles, tokens, invites, second factors, audit writes
☐ 8 Registry, egress check, connections, health
☐ 9 Mock, OpenAI-compatible and Anthropic modules pass conformance
☐ 10 Workspaces, roles, task types, templates, builder, versions, export
☐ 11 Reserve and settle at three levels, ledger, estimates
☐ 12 Orchestrator: plan, role steps, retries, approvals, cancel, recovery
☐ 13 Outbox, relay, bus with replay, WebSocket hub
☐ 14 Inbox, quiet hours, push, digest
☐ 15 API with a guard on every route, problem details, sign-in providers
☐ 16 Web shell, tokens, UI kit, core screens, accessible
☐ 17 Node editor with keyboard, undo, validation
☐ 18 Tool port, MCP dock, tool steps
☐ 19 If, router, merge, text, stop, connectors, loops
☐ 20 Workspace steps with repeats, guards and budget shares
☐ 21 Agent and Workflow ports, waiting across restarts, questions, callbacks
☐ 22 CMP, remote and sidecar docks, webhook dock, sample agent
☐ 23 Local Bridge: pairing, signed calls, default deny
☐ 24 A2A dock for 1.0 and 0.3
☐ 25 Installed modules from a folder
☐ 26 Trace: one id per tree, spans, timeline screen
☐ 27 Insights and audit trail
☐ 28 Postgres suite, smoke, accessibility, CI, Docker
☐ 29 Rules: allow, ask first, never, inherited down the nest
☐ 30 Roadmap items, in order, behind the product gates